

AI AGENT 学习路线 · 学习专题 009

Agent 框架实战 学习笔记（打印版）

11 个篇章 · 7 张矢量架构图

生成日期：2026-09-19 | 由 Markdown 笔记自动排版生成

目录

01

009 · Agent 框架实战 导航

02

00 · 背景篇：为什么会有这么多框架 含架构图 ✓

03

frameworks 章节导航

04

01 · 问题篇：框架到底帮你省了什么

05

02 · 概念篇：框架核心概念精讲（12 张概念卡） 含架构图 ✓

06

03 · 原理篇：框架共性架构解剖 含架构图 ✓

07

04 · 进阶篇：持久化与 Human-in-the-Loop 含架构图 ✓

08

05 · 实战篇：LangGraph 重写点价 Agent（全代码） 含架构图 ✓

09

06 · 生产篇：可观测、评估与部署

10

07 · 对比篇：框架选型决策 含架构图 ✓

11

08 · 自测篇：题库与面试题 分级题库 ✓

共 11 个篇章。

009 • Agent 框架实战

主题定位：把手写 Agent 循环升级为**框架驱动**。以 LangGraph 重写 003 篇的点价 Agent 为主线，讲透框架的共性架构、持久化、Human-in-the-Loop、可观测与选型决策。

读者画像：中基集团 CTRM 贸易系统开发者（Java 11 + Vue 2.7 + MySQL）。实战代码以 Python 为主，每章附 Java 生态对照（Spring AI / LangChain4j），方便平移到现有技术栈。

与前面章节的衔接

前置章节	本文承接点
003 · Agent 从零实现	本文把 003 手写的"路由→抽取→查库存→查价→生成建议单→人工确认"循环，逐件映射到 LangGraph 的工程件上，实战篇给完整重写代码
004 · RAG	04 篇的检索件在框架里通常作为一个节点接入，本文 06 篇讲评估时沿用 004 的检索评估思路
006 · 评估与可观测	06 生产篇沿用 006 的评估集纪律，叠加框架自带的 eval 与 trace 设施
007 · 多 Agent 协作	04 进阶篇用 subgraph 把 007 的多 Agent 拓扑落进 LangGraph，对比篇也覆盖 CrewAI/AutoGen 的多 Agent 路线

目录树

```
009-agent框架实战\
├── README.md                ← 本文件：导航
├── 00-背景篇-为什么会有这么多框架.md
├── frameworks\
│   ├── README.md           ← frameworks 章节导航
│   ├── 01-问题篇-框架到底帮你省了什么.md
│   ├── 02-概念篇-框架核心概念精讲.md      (12 张概念卡)
│   ├── 03-原理篇-框架共性架构解剖.md
│   ├── 04-进阶篇-持久化与HumanInLoop.md
│   ├── 05-实战篇-LangGraph点价Agent.md     (Python 全代码)
│   ├── 06-生产篇-可观测评估与部署.md
│   ├── 07-对比篇-框架选型决策.md
│   └── 08-自测篇-题库与面试题.md
├── diagrams\
│   ├── 009-01-框架演进时间线.svg
│   ├── 009-02-框架共性架构解剖.svg
│   ├── 009-03-框架核心概念地图.svg
│   ├── 009-04-手写vs框架工程件对照表.svg
│   ├── 009-05-点价Agent状态图.svg
│   ├── 009-06-HumanInLoop中断恢复时序.svg
│   └── 009-07-六框架选型对比.svg
```

学习路线（建议顺序）

1. **建立问题意识**：00 背景篇 → 01 问题篇。搞清"框架到底省了什么、没省什么"。
2. **攒概念**：02 概念篇 12 张概念卡，配 009-03 概念地图反复看。
3. **看穿共性**：03 原理篇。所有框架都是"状态容器 + 调度循环 + 工具运行时 + 持久化钩子 + 可观测钩子"五件套。
4. **上手重写**：04 进阶篇（持久化与 HITL 原理）→ 05 实战篇（点价 Agent 全代码，边读边跑）。
5. **走向生产**：06 生产篇（trace / 评估 / 部署）。

6. **会选型**：07 对比篇决策树 + 六框架大表；08 自测篇检验。

7. **Java 团队**：每章末尾的 Spring AI / LangChain4j 对照段落不要跳过；07 对比篇有专门的 Java 选型结论。

阅读约定

- 每章开头【本节问题】给出 4 个驱动问题；结尾【本节自测】5~8 题。
- 核心概念一律【一句话人话】+【生活类比】（费曼式）。
- 时效性事实均标注来源与日期，检索不到的标注"截至 2026-09 未见权威披露"。
- 贯穿案例：**点价 Agent**（CTRM 场景：客户报盘 → 路由 → 抽取 → 查库存 → 查价 → 生成建议单 → 交易员人工确认）。

00 · 背景篇：为什么会有这么多框架

【本节问题】① Agent 框架为什么在 2023~2026 短短四年爆发式增长？② "框架太多"的真实原因是什么？③ 框架存在的本质价值是什么——没有框架就做不了 Agent 吗？④ 作为 CTRM 开发者，该如何看待这场框架混战？

1. 框架爆发时间线

先看一张图建立时间感（细节后文逐条展开）。



2023 · 胶水时代：LangChain agents 一统注意力

- 2022-11 ChatGPT 发布，2023 年上半年 Agent 概念爆发。LangChain 的 AgentExecutor + ReAct prompt 成了事实标准——本质是"LLM 循环 + 工具 JSON 解析"的胶水库。
- 同期 AutoGen (Microsoft Research, 2023-10 论文) 提出多 Agent 对话范式；MetaGPT 等角色扮演框架扎堆出现。
- 痛点显形：AgentExecutor 把控制流藏在黑盒 prompt 解析里，不可控、难调试、循环无界。社区怨声最大的三件事：**状态不可见、循环不可控、出错不能恢复。**

2024 · 状态机时代：把 Agent 画成图

- 2024-01 LangGraph 首次发布，作为 LangChain 生态的图编排扩展；2024-06 v0.1 稳定版引入生产级特性（持久化、Human-in-the-Loop）。（来源：博客园《LangGraph：图驱动的有状态智能体编排运行时》，2026-09 检索）
- 核心转向：**Agent 不是 prompt 循环，是状态机。**节点即步骤，边即转移，状态显式可见——正好回应 2023 年三大痛点。
- CrewAI (2024 年初爆红) 反其道而行：不画图，用 Role/Goal/Backstory 的角色隐喻降低心智负担，主打"像带团队一样编排 Agent"。2024-10 宣布 1800 万美元融资（来源：aiwiki.ai，2026-09 检索）。

2025 • 正式版元年：各家冲 1.0，官方下场

- 2025-03 OpenAI 发布 **Agents SDK**，替代实验性 Swarm，提供 tool use、handoffs、guardrails、tracing 四件生产积木（来源：aiwiki.ai，2026-09 检索）。
- 2025-05 **Spring AI 1.0 GA**，Java 生态等来了官方答案；1.1 加入 MCP 与 agent 能力（来源：fast.io《Best AI Agent Frameworks for Java in 2026》）。
- 2025-10-01 Microsoft 公开预览 **Microsoft Agent Framework**（Semantic Kernel + AutoGen 合体），AutoGen 本体进入维护模式（来源：aiwiki.ai，2026-09 检索）。
- 2025-10-22 **LangGraph 1.0 GA**，与 LangChain 1.0 同步，官方称"持久化 Agent 框架首个稳定版"；生产用户含 Uber、LinkedIn、Klarna（来源：LangChain 博客转载、aiwiki.ai）。

2026 • 分化沉淀：格局收敛

- LangGraph 迭代至 1.2.x，新增 DeltaChannel、命令式 API 等（来源：博客园，2026-09 检索）；LangChain + LangGraph 合计月下载 9000 万，35% 的 Fortune 500 在用（来源：LangChain 1.25 亿美元融资公告）。
- Microsoft Agent Framework 于 2026-04-03 GA 1.0（.NET / Python）（来源：aiwiki.ai）。
- CrewAI 1.15.x 主打 Flows 工作流；Spring AI 2.0 GA（Spring Boot 4 兼容 + A2A 协议）；LangChain4j 1.15→1.18，新增 langchain4j-agentic 模块（来源：CSDN《十六个主流AI应用开发框架全面对比(2026年8月版)》《LangChain4j vs Spring AI》）。
- **格局判断**：不再有"新框架"的爆发，转而沉淀为几个稳定梯队：图编排（LangGraph / MAF）、轻量官方 SDK（OpenAI Agents SDK）、角色协作（CrewAI）、Java 生态（Spring AI / LangChain4j）。

2. "框架太多"的真实原因

费曼一下：所有 Agent 本质都在干同一件事——

```
while (任务未完成):  
    让 LLM 看着当前状态，决定下一步（调工具 / 换话题 / 结束）  
    执行这一步，把结果写回状态
```

就这三行循环，为什么能冒出几十个框架？因为**循环、状态、工具这三样东西的"胶水层"各有取舍**，每个取舍维度都能切出一个新框架：

取舍维度	一头	另一头	代表框架
控制流	显式图（看得见每条边）	隐式循环（LLM 自主决定）	LangGraph ↔ OpenAI Agents SDK
状态	显式共享 State（强类型）	消息流（对话即状态）	LangGraph ↔ AutoGen/AG2
抽象心智	状态机（工程隐喻）	角色/团队（组织隐喻）	LangGraph ↔ CrewAI
语言生态	Python 优先	JVM 优先	LangChain ↔ Spring AI / LangChain4j
持久化	内置一等公民（checkpointing）	留给开发者	LangGraph ↔ 早期 CrewAI
厂商绑定	中立	官方绑定自家模型	LangChain4j ↔ OpenAI Agents SDK

一句话：框架多不是因为缺乏共识，而是共识（循环+状态+工具）之上的工程取舍维度太多。每个框架赌的是"哪几件胶水最值得官方帮你粘好"。

给 CTRM 开发者的映射：你们写 Java 时为什么不觉得"Web 框架太多"？因为 Spring MVC 和 JAX-RS 在十年前已经完成了收敛。Agent 框架正走在 2026 年的收敛路上——**先懂共性 (03 原理篇)**，框架只是壳。

3. 框架的本质价值：不写会死，写了救命的五件事

没有框架，003 篇你已经手写出一个能跑的点价 Agent。框架不会让 Agent 变聪明，它救的是工程命：

- 1. **状态容器**：显式、可类型化、可校验的状态对象，替代手搓 dict 传递。
- 2. **调度循环**：路由、分支、并行、重试的编排，替代手写 while + if。
- 3. **工具运行时**：工具注册、schema 校验、超时/异常处理的统一层。
- 4. **持久化钩子**：每步落盘（checkpoint），崩溃恢复、暂停恢复、时间旅行——**手写最难、框架最值钱的一件。**
- 5. **可观测钩子**：trace/span 自动埋点，接 LangSmith / Langfuse / OTel。

反过来，框架**不帮你省**的：业务逻辑本身、prompt 质量、评估数据集、领域知识。这就是 01 问题篇的主题。

4. CTRM 视角：为什么本篇选 LangGraph 做贯穿案例

点价 Agent 的业务特征和 LangGraph 的能力——对应：

点价业务需求	LangGraph 对应件
报盘类型不同走不同流程	条件边（router）
抽取结果要结构化可校验	显式 State（TypedDict）
高危动作必须交易员确认	interrupt / interrupt_before
建议单挂起数小时等人审	checkpointer 持久化中断
审计要求每步可回溯	每步 checkpoint + trace
未来可能拆多 Agent（套保/风控）	subgraph 组合（联动 007）

Java 对照：Spring AI 1.0 GA 后 Java 团队终于有官方框架，但其编排能力仍在追赶 LangGraph；LangChain4j 的 agentic 模块则提供了 Workflow 与声明式 AI Services 两种路线（来源：CSDN 2026-07 对比文）。07 对比篇会给出 Java 团队选型结论，这里先记住：**用 LangGraph 学概念，用 Spring AI / LangChain4j 落 JVM**，两边的概念 90% 同构。

【本节自测】

- 1. 2023→2026 框架演进的主线是什么？（要点：胶水 AgentExecutor → 状态机/图 → 各家 1.0 正式版 → 分化沉淀为稳定梯队）
- 2. 为什么说“框架多”是因为胶水层取舍维度多？至少说出 3 个维度。（要点：控制流显隐、状态形态、抽象心智、语言生态、持久化、厂商绑定）
- 3. LangGraph 1.0 GA 的时间与定位？（要点：2025-10-22，与 LangChain 1.0 同步，官方称持久化 Agent 框架首个稳定版）
- 4. OpenAI Agents SDK 的出身与四件核心积木？（要点：2025-03 替代 Swarm；tool use / handoffs / guardrails / tracing）
- 5. AutoGen 2026 年的现状？（要点：进入维护模式，与 Semantic Kernel 合并为 Microsoft Agent Framework，MAF 2026-04-03 GA）
- 6. 框架不帮你省哪些事？（要点：业务逻辑、prompt 质量、评估集、领域知识）
- 7. 点价 Agent 的哪三个业务特征最依赖框架？（要点：高危动作人审（interrupt）、长挂起（checkpoint）、每步可回溯（checkpoint+trace））

frameworks 章节导航

本目录按"问题 → 概念 → 原理 → 进阶 → 实战 → 生产 → 对比 → 自测"的顺序拆解 Agent 框架。贯穿案例：用 LangGraph 重写 003 篇点价 Agent。

章节	一句话	建议耗时
01-问题篇	算清楚框架帮你省了什么、没省什么	20 min
02-概念篇	12 张概念卡，框架词汇表	45 min
03-原理篇	五层共性件解剖 + 手写对照表	30 min
04-进阶篇	checkpointer、interrupt、store、subgraph	40 min
05-实战篇	点价 Agent 全代码重写	60 min
06-生产篇	trace、评估、部署、升级坑	30 min
07-对比篇	决策树 + 六框架大表 + Java 结论	25 min
08-自测篇	题库与面试题总汇	自定

配图集中在 `../diagrams/`，7 张图与章节对应关系：

- 009-01 时间线 → 背景篇 (00)
- 009-02 共性架构 → 03 原理篇
- 009-03 概念地图 → 02 概念篇
- 009-04 工程件对照 → 03 原理篇
- 009-05 点价状态图 → 05 实战篇
- 009-06 HITL 时序 → 04 进阶篇
- 009-07 六框架对比 → 07 对比篇

01 · 问题篇：框架到底帮你省了什么

【本节问题】① 003 篇手写的点价 Agent 已经能跑，为什么还要上框架？② 框架帮你省的到底是什么——代码量、正确性还是可维护性？③ 哪些事框架永远帮不了你？④ Java 工程师用什么类比理解"框架换走的是控制权"？

1. 先复盘 003 篇：手写点价 Agent 的工程件清单

003 篇用纯 Python 手写了点价 Agent：路由 → 抽取 → 查库存 → 查价 → 生成建议单 → 人工确认。把代码里的"工程件"拆出来数一数：

手写工程件	003 篇实现方式	痛点
状态传递	Python dict 层层传参	字段一多就乱，无类型校验
路由分发	if/elif 判断意图	规则散落，加一类报盘改三处
工具调用	手写函数 + 手拼 JSON prompt	schema 无校验，异常各写各的
循环上限	<code>while step < MAX_STEPS</code>	忘写就死循环烧 token
人工确认	<code>input()</code> 阻塞	进程挂住、重启全丢
日志	print / logging 手埋	链路不连贯，事后查不了
失败恢复	没有	崩了从头再来

手写版能跑，但每一步都是你欠的债。框架的本质：把这 7 件债变成官方还过的债。

2. 框架帮你省的三层东西

【一句话人话】框架省的不是"代码行数"，省的是"你没想到的坏情况"。

【生活类比】手写 Agent 像自己装修水电——能亮灯能出水；用框架像买精装房——水电验收标准是行业写好的，你踩不到"零线火线接反"这种坑。

第一层：省样板代码（最薄的一层）

工具注册带 schema 校验、消息拼装、流式输出管道。这一层价值有限——003 篇证明你手写也就半天。只冲这层上框架，不值。

第二层：省控制流的正确性（中间层）

路由、分支、并行、循环上限、错误转移——框架把这些变成"声明"而不是"代码"。比如 LangGraph 里条件边是图上一条线，视觉可查、改动局部化；手写版是散在 router 函数里的 if。CTRM 对照：这就是把业务流程从散装 if 升级成 BPMN 流程图的工程收益，你们做贸易审批流时体会过。

第三层：省状态与恢复的可靠性（最值钱的一层）

手写版进程一崩、input() 一挂、部署一重启，Agent 状态全丢。框架的 checkpointer 让每一步自动落盘：崩溃从最近 checkpoint 恢复、人工确认可以挂起几小时甚至几天、改错还能时间旅行回滚。这一层手写要付出"消息队列 + 状态库 + 恢复协议"的全套代价——框架 80% 的价值在这层。

优先级要点：选框架先看第三层（持久化与人审），再看第二层（控制流表达力），第一层只是顺手。

3. 框架永远帮不了你的四件事

1. **业务规则本身**：点价窗口期校验、基差报价合规规则——框架不知道什么是"点价"。
 2. **Prompt 质量**：抽取报盘要素的准确率取决于 prompt + 模型，不取决于 StateGraph。
 3. **评估数据集**：框架自带 eval 工具（06 篇），但评估集纪律要靠 006 篇的 methodology 自己建。
 4. **领域边界设计**：哪些动作该人审、哪些可自动，是风控决策，不是框架决策。
- 费曼检验**：如果把框架从项目里删掉，你的业务逻辑应该一行都不用改——改不了的，说明你把业务逻辑泄漏进框架了，这是 05 实战篇反复强调的写法纪律。

4. Java 工程师类比：框架 = Spring

你对"框架换控制权"其实早有体感：

Java 世界	Agent 框架世界	共同代价
手写 Servlet	手写 Agent 循环	全都要自己来
Spring IoC 接管对象装配	LangGraph 接管执行流转	控制权交给框架，换可维护性
Spring 事务管理	checkpointer 持久化	声明式 vs 手写 commit
Actuator / Micrometer	trace/span 自动埋点	统一可观测口径
换框架 = 伤筋动骨	LangChain 0.x→1.0 断代	锁定生态的风险

Spring AI 的定位正是"把 LLM 当成新一代 JDBC"——Spring 三十年来干的都是"把异构系统抽象成可移植接口"，LLM 只是新加入的抽象对象（来源：守株阁《Spring AI 深度》，2026-05）。所以 Java 团队学框架概念的最快路径：**先认共性件（03 篇五层解剖），再映射回 Spring 心智。**

【本节自测】

1. 手写点价 Agent 的 7 个工程件是什么？（要点：状态传递/路由/工具调用/循环上限/人工确认/日志/失败恢复）
2. 框架省的三层东西，哪层最值钱，为什么？（要点：状态与恢复可靠性；崩溃恢复+长挂起+时间旅行手写代价最高）
3. 框架帮不了的四件事？（要点：业务规则、prompt 质量、评估集、领域边界设计）
4. "业务逻辑泄漏进框架"如何检验？（要点：删框架业务逻辑应零改动）
5. checkpointer 类比 Spring 的什么机制？（要点：声明式事务/持久化，自动落盘替代手写 commit）
6. Java 团队为什么不建议只冲"省样板代码"上框架？（要点：第一层价值薄，手写半天可得；应看控制流与可靠性层）
7. Spring AI 的官方使命类比是什么？（要点：把 LLM 抽象成新一代 JDBC/新中间件）

02 · 概念篇：框架核心概念精讲（12 张概念卡）

【本节问题】① LangGraph 的 StateGraph 到底是什么，和流程引擎什么关系？② checkpoint 与 interrupt 为什么是"一对搭档"？③ command、handoff、guardrail 各解决什么问题？④ 这些概念在 Spring AI / LangChain4j 里有对应物吗？

先看全景地图，12 张概念卡按图上的四组展开。

009 · Agent 框架核心概念地图（12 卡）

四组概念：图的骨架 · 运行保障 · 协作扩展 · 记忆观测 —— 配合 02 概念篇逐卡精读

① 图的骨架（控制流怎么画）

StateGraph 状态图

显式状态图 · 允许环
共享状态对象流动

节点 / 边

节点=函数(读状态返增量)
边=固定走向 START/END

条件边 Router

运行时按状态动态分支
替代散装 if/elif

Command

goto + update 打包
节点内局部跳转-1.x 新API

② 运行保障（可靠性三件）

checkpointer

每步自动存档·崩溃恢复
time-travel 回放

interrupt 中断

人工审批·挂起数天可续
与 checkpoint 是搭档

streaming 流式

token 级（打字机） / 事件级 updates（进度条） / 状态级
生产最常用 updates：每节点完成推一次增量

③ 协作扩展（多Agent与安全）

handoff 移交

多Agent间控制权转交
客服转接类比

subgraph 子图

图当节点复用·分层组合
007 多Agent落地处

guardrail 护栏

输入/输出并校验关卡：不合规拦截·改写·转人工
点价例：数量为正·价格在限价带内

④ 记忆观测（长期支撑）

memory 记忆

短期=thread 状态历史
长期=Store 跨会话

trace / span

trace=全链路·span=分段
一次节点执行=一个span

贯穿案例：点价 Agent

用到 StateGraph · Router 条件边 · interrupt 人审
SqliteSaver 持久化 · stream updates 进度推送

记忆口诀：骨架四件画图，保障三件保命，协作两件扩多Agent，记忆观测撑长期 —— 先记组再记卡

卡 1 · StateGraph（状态图）

【一句话人话】用一张显式的图来定义 Agent 的所有步骤和步骤之间的走向，图上流动的是一个共享状态对象。

【生活类比】工厂流水线的工序图：每个工位（节点）从传送带（状态）上取料、加工、放回去；传送带去哪个工位由轨道（边）决定。

- State 用 TypedDict 或 Pydantic 定义（LangGraph 惯例），字段显式、可校验。
- 与 BPMN/工作流引擎同构：CTRM 开发者可以把 StateGraph 理解为"给 LLM 步骤用的 Activiti"。
- 之所以能成为 Agent 的底座：图允许环（LLM 不确定要几轮），这是与 DAG 流程引擎的关键差异。

卡 2 · 节点（Node）与边（Edge）

【一句话人话】节点是一段接收状态、返回状态更新的函数；边是节点之间的固定走向。

【生活类比】节点是函数，边是 `return next()` 的调用关系表——只是把调用关系从代码里抽出来摆到图上。

- 节点签名：`def node(state) -> dict`，返回值是增量更新（patch），框架负责合并进共享状态。
- 固定边 `add_edge(a, b)`：无条件走向；起点 `START`、终点 `END` 是内置哨兵。

- 并行：多个边从同一节点扇出（fan-out），框架自动并行执行；用 reducer 指定并发结果如何合并。

卡 3 • Router（条件边）

【一句话人话】条件边是在运行时根据当前状态动态决定“下一步去哪个节点”的分支函数。

【生活类比】快递分拣机：包裹（状态）经过时，扫码器读地址决定送上哪条传送带。

```
graph.add_conditional_edges("router", route_fn, {"price": "查价", "stock": "查库存", "end": END})
```

- `route_fn(state) -> key`，映射表把 key 指到目标节点。
- 这就是 003 篇手写 `if intent == ...` 的框架化对应物——分支逻辑从散装 if 变成图上一条可审查的线。

卡 4 • Command (goto + update)

【一句话人话】LangGraph 1.x 里节点返回值可以直接携带“状态更新 + 下一步去哪”的打包指令。

【生活类比】快递员不但把包裹放上传送带，还顺手在面单上写了“下一站直接送 A 区”。

```
from langgraph.types import Command
def node(state) -> Command:
    return Command(update={"price": 8120.0}, goto="生成建议单")
```

- 把“路由函数 + 节点函数”合一，适合分支逻辑与该节点强耦合的场景。
- 与条件边的关系：条件边是图上显式的线（全局可见），Command 是节点内的隐式跳转（局部灵活）。

卡 5 • Checkpointer（断点恢复）

【一句话人话】在每一步执行后把整个图状态自动落盘的机制，崩溃可恢复、中断可续跑、历史可回放。

【生活类比】游戏存档：每个关卡打完自动存档，死了从存档重来，还能读旧档换打法。

- 后端可选：内存（测试）、SQLite（单机/开发）、Postgres（生产）。
- 以 `thread_id` 为键组织存档，一个 thread 就是一次完整会话/一单业务。
- 附赠 time-travel：任意历史 checkpoint 可回放、改状态、分叉重跑。
- Java 对照：等价于“自动给每一步做 DB 事务快照”，但框架帮你把快照、序列化、恢复协议全做了。

卡 6 • Interrupt（人工中断）

【一句话人话】让图执行到指定位置主动暂停、把控制权交给人类的机制；人确认后用 resume 恢复执行。

【生活类比】采购审批流：金额超过 50 万的订单流程自动停在“待总经理审批”节点，审批人签字后流程继续。

- 两种用法：编译时 `interrupt_before=["高危节点"]`；运行时在节点内 `interrupt(payload)` 抛出。
- 与 checkpointer 是一对搭档：中断之所以能“挂起几天再恢复”，靠的是状态已落盘。
- 点价场景：生成建议单后必须交易员确认——这是 003 篇手写 `input()` 的生产级替代（详见 04 进阶篇 + 009-06 时序图）。

卡 7 • Handoff（控制权移交）

【一句话人话】多 Agent 场景里，把对话/任务的控制权从一个 Agent 转交给另一个 Agent 的模式。

【生活类比】客服转接：一线客服解决不了，转接给售后专家，客户无感但接线人换了。

- OpenAI Agents SDK 把 handoff 做成一级原语 (`transfer_to_xxx` 工具) ; LangGraph 里通常用条件边/Command + 子图实现同等效果。
- 与 007 篇多 Agent 拓扑的关系: handoff 是"星型/主管型"协作里的关键动作。

卡 8 • Guardrail (护栏)

【一句话人话】在 Agent 输入或输出上并联一道校验关卡, 不合规就拦截/改写/拒答。

【生活类比】高速收费站 ETC: 不合规车辆(超载、未装卡)在入口就被拦下, 不进主路。

- OpenAI Agents SDK 内置 guardrails (并行跑在 agent 外侧) ; LangGraph 里通常实现为图上的校验节点, 失败走条件边转人工或重试。
- 点价场景: 抽取结果校验(数量为正、价格在限价带内)就是最典型的业务 guardrail。

卡 9 • Trace / Span (追踪)

【一句话人话】trace 是一次完整执行的全链路记录, span 是链路中的一段(一次 LLM 调用、一次工具执行)。

【生活类比】物流单号(trace)与每一段"已发出→已到转运中心→派送中"的记录(span)。

- 框架自动埋点: LangGraph 每个节点执行天然是一个 span。
- 后端: LangSmith (同门)、Langfuse (开源可自托管)、OTel 标准协议。06 生产篇展开。

卡 10 • Memory (短期线程 / 长期存储)

【一句话人话】短期记忆是单个 thread 内的状态历史(靠 checkpointer 天然获得), 长期记忆是跨 thread 持久化的知识(LangGraph 用 Store)。

【生活类比】短期记忆=本次工作会话的草稿纸; 长期记忆=你的工作笔记本, 第二天上班还带着。

- 短期: `state["messages"]` + checkpointer = 多轮对话上下文自动续。
- 长期: InMemoryStore / Postgres Store, 按 namespace+key 存取, 跨 thread 共享。
- 点价场景: 客户的历史报价偏好、上次砍价记录→长期记忆; 本单报盘的上下文→短期。

卡 11 • Subgraph (子图)

【一句话人话】把一张图整体作为另一个图的节点复用, 实现模块化与多 Agent 分层组合。

【生活类比】公司组织架构: 事业部(子图)对集团(父图)来说就是一个部门节点, 内部怎么运作集团不关心。

- 父子图通过状态字段映射通信; 007 篇的 supervisor 模式在 LangGraph 里最自然的实现就是"主管图 + 各专长子图"。
- CTRM 场景: 点价子图、套保子图可以做成独立 subgraph, 由风控主管图调度。

卡 12 • Streaming (token / 事件流)

【一句话人话】把执行过程实时推给调用方: token 级(逐字)、事件级(每步完成推一次)、状态级(状态变更推送)。

【生活类比】外卖骑手轨迹页: 不是等送达才告诉你结果, 而是每过一个路口更新一次位置。

- token 流: LLM 生成逐字输出, 前端打字机效果 (Vue 2.7 前端可用 SSE 对接)。
- 事件/节点流: `graph.stream(..., stream_mode="updates")`, 每个节点完成推一次增量——生产上最有用的是"进度可见"。
- 与 checkpointer 组合: 中断/恢复事件也走流, 前端能感知"已挂起待审批"。

Java 生态对照段

本篇概念	Spring AI 对应	LangChain4j 对应
状态/消息流	ChatMemory + Advisor 链	ChatMemory / AgenticScope (agentic 模块)
条件路由	无原生 (Advisor 手写分支)	agentic 模块 Workflow 编排
持久化恢复	无内置 (自己落库)	无内置, 社区扩展
interrupt 人审	无原生	无原生
工具	@Tool / FunctionCallback	@Tool 注解
guardrail	Advisor 前后置校验	Guardrails API (1.x)
trace	Micrometer / OTel 原生	OTel 支持

(依据: CSDN《LangChain4j vs Spring AI》2026-07、fast.io 2026、守株阁 2026-05。共同短板: **JVM 框架目前都没有 LangGraph 式内置持久化中断**, Java 团队要 HITL 得自己落库或把编排层交给 Python 服务——07 对比篇会展开。)

【本节自测】

1. StateGraph 与 DAG 流程引擎的关键差异? (要点: 允许环, LLM 步数不定)
2. 节点返回 dict 的语义是什么? 谁负责合并? (要点: 增量 patch, 框架按 reducer 合并进共享状态)
3. 条件边和 Command 各适合什么场景? (要点: 条件边=全局显式分支; Command=节点内局部跳转+更新打包)
4. 为什么说 checkpointer 与 interrupt 是搭档? (要点: 中断挂起靠状态已落盘, 恢复靠读档)
5. checkpointer 的三种后端与适用环境? (要点: 内存/测试、SQLite/开发单机、Postgres/生产)
6. handoff 与 subgraph 分别解决什么? (要点: handoff=控制权转交; subgraph=模块化分层复用)
7. 短期记忆和长期记忆的实现差异? (要点: thread 内状态历史 vs Store 跨 thread; 点价例子)
8. trace 和 span 的关系, LangGraph 里 span 的天然单位? (要点: 全链路 vs 分段; 一个节点执行=一个 span)
9. streaming 三种粒度? (要点: token/事件/状态; 生产最常用 updates 事件流)
10. JVM 框架在框架概念上的共同短板? (要点: 无内置持久化中断/HITL, 编排能力仍在追赶)

03 • 原理篇：框架共性架构解剖

【本节问题】① 剥掉 API 外壳，所有 Agent 框架的共同骨架是什么？② 003 篇手写循环的每个工程件，在框架里的对应物是什么？③ 五层共性件之间如何协作形成执行闭环？④ 为什么懂了共性件，换框架就只是"换语法"？

1. 五层共性件：所有框架的公约数

不管 LangGraph、OpenAI Agents SDK、CrewAI 还是 Spring AI，剥掉外壳后都在实现同一副骨架：

009 • Agent 框架共性架构解剖（五层件）

所有框架 = 这五层的不同取舍实现；理解五层，换框架只是换语法

① 状态容器

定义流动数据的 schema 与合并规则（reducer）

形态分化：

显式 State（LangGraph）
↔ 消息流（AutoGen）/ 隐式上下文（CrewAI）

② 调度循环

决定下一步执行谁：顺序 / 分支 / 并行 / 终止

形态分化：

图引擎（LangGraph/MAF）· LLM自主循环（AgentsSDK）· 角色剧本（CrewAI）

③ 工具运行时

工具注册 · schema校验 · 超时重试 · 结果回填

统一趋势：

MCP 协议已成事实标准（2026）
Spring AI 1.1+ / LangChain4j 1.x 原生支持

④ 持久化钩子 ★ 分野核心

checkpoint · thread 组织 · resume 协议 · time-travel

差距最大的层：

LangGraph 内置三后端（内存/SQLite/Postgres）
JVM 框架与多数 SDK 均无内置 → 手写代价最高

⑤ 可观测钩子

trace/span 自动埋点 · token 统计 · callbacks

统一趋势：

OpenTelemetry 最大公约数
LangSmith / Langfuse 在其上做产品化

执行闭环示例

以点价 Agent「查价」段为例

调度循环选中节点「查价」

读入共享状态 state

工具运行时调用接口

增量 patch 合并进状态

持久化钩子写 checkpoint

可观测钩子结束 span 上报

下一节点 / interrupt 挂起

五件齐了，Agent 才从 demo 变成系统：状态记什么 · 循环按什么顺序 · 工具怎么干活
干到哪记住哪 · 干了什么看得见

【一句话人话】每个框架都是一台"自动贩卖机"：货道是工具运行时，投币记录是状态容器，出货控制板是调度循环，断电续卖是持久化钩子，后台流水是可观测钩子。

第 1 层 • 状态容器

- 职责：定义"Agent 执行过程中流动的数据"的 schema、合并规则（reducer）、校验。
- 形态分化：LangGraph 显式 TypedDict/Pydantic State；OpenAI Agents SDK 用消息列表 + run context；CrewAI 用任务间隐式上下文传递；AutoGen/AG2 用对话消息流。
- 取舍点：状态越显式，可审计性和可控性越强，样板越多——这就是 00 背景篇"胶水层取舍"的第一维。

第 2 层 • 调度循环

- 职责：决定"下一步执行谁"——顺序、分支、并行、循环、终止条件。
- 形态分化：图引擎（LangGraph / MAF workflow）、LLM 自主循环（OpenAI Agents SDK 的 run loop、经典 ReAct）、角色剧本（CrewAI crew/process）、对话轮转（AutoGen group chat）。
- 循环上限（recursion limit / max_turns）是所有框架的标配安全阀。

- 15 / 37 -

第 3 层 · 工具运行时

- 职责：工具注册 (name/description/参数 schema)、调用分发、超时与异常包裹、结果回填。
- 共性趋势：MCP 协议 2025~2026 成为工具接入事实标准——Spring AI 1.1+、LangChain4j 1.x 均已原生支持（来源：CSDN、fast.io, 2026）。
- CTRM 对照：工具运行时 = 你的 Service 层注册表 + 统一 AOP 拦截（超时/重试/审计）。

第 4 层 · 持久化钩子

- 职责：状态快照 (checkpoint)、线程组织 (thread)、恢复协议 (resume)、可选时间旅行。
- 差距最大的层：LangGraph 内置 checkpointer 三后端；OpenAI Agents SDK / CrewAI 需自建或借外部（如 Temporal）；JVM 框架基本没有内置。
- **这层是框架分野的核心**：能不能“挂起几天等人审再续跑”，取决于这层。

第 5 层 · 可观测钩子

- 职责：trace/span 埋点、token 统计、回调接口 (callbacks)。
- 共性趋势：OpenTelemetry 成为最大公约数；LangSmith、Langfuse 在其上做产品化。

2. 手写循环 → 框架工程件对照表

把 003 篇手写点价 Agent 的每一件，映射到 LangGraph 的对应物——这张表是本篇的通关凭证：

009 · 003 手写循环 vs LangGraph 工程件对照				
行 6~8 为「能力从无到有」（橙底高亮）—— 框架最值钱的部分；行 1~5 只是表达力升级				
#	手写工程件	003 篇手写实现	LangGraph 对应物	框架多了什么
1	状态传递	dict 层层传参	State(TypedDict) + reducer	类型化 · 校验 · 并发合并
2	路由分发	if/elif 关键词匹配	条件边 / Command	分支上图画 · 改动局部化
3	业务步骤	一堆 handler 函数	节点（统一签名）	自动埋点 · 可单独测试
4	工具调用	手拼 JSON + 各写 try	工具运行时 (@tool)	schema 校验 · 统一超时重试
5	循环与终止	while + MAX_STEPS	图的环 + recursion_limit	引擎兜底 · 忘写也不死循环
6	人工确认	input() 阻塞进程	interrupt / resume	可挂起数天 · 可改状态再恢复
7	断点恢复	(无)	checkpointer (3 后端)	崩溃恢复 · time-travel
8	日志追踪	print / logging 手埋	trace / span 自动埋点	全链路 · 接 LangSmith/Langfuse
9	输出推送	函数一次性返回	stream (token / 事件)	前端实时进度 · SSE 对接
结论：1~5 表达力升级（能写的写法变优雅）；6~8 能力从无到有（手写做不起）；9 体验加分				
05 实战篇每段代码回扣本表行号 —— 逐段对账				

#	003 篇手写件	手写实现	LangGraph 对应	框架多了什么
1	状态传递	dict 传参	State (TypedDict) + reducer	类型化、校验、并发合并规则
2	路由分发	router 函数 if/elif	条件边 / Command	分支上图画，可视化、局部化修改
3	业务步骤	一堆 handler 函数	节点	统一签名、自动埋点、可单独测试

#	003 篇手写作	手写实现	LangGraph 对应	框架多了什么
4	工具调用	手拼 JSON + try/except	工具运行时 (@tool / ToolNode)	schema 校验、统一超时重试
5	循环与终止	while + MAX_STEPS	图的环 + recursion_limit	声明式，引擎兜底
6	人工确认	input() 阻塞	interrupt / interrupt_before	可挂起数天、可改状态后恢复
7	断点恢复	无	checkpoint	崩溃恢复、time-travel
8	日志追踪	print/logging	trace/span 自动埋点	全链路、接 LangSmith/Langfuse
9	输出推送	无 (函数返回)	stream (token/事件)	前端实时进度

读表结论：1~5 是"表达力升级" (能写的写法变优雅)，6~8 是"能力从无到有" (手写根本没做或做不起)。05 实战篇每段代码都会回扣这张表的行号。

3. 执行闭环：五层件如何协作

以点价 Agent"查价 → 生成建议单 → 中断等人审"这一段为例走一遍闭环：

调度循环(第2层)选中节点"查价"

→ 从状态容器(第1层)读入 state

→ 节点内经工具运行时(第3层)调用接口

→ 返回增量 patch, reducer 合并进 state

→ 持久化钩子(第4层)自动写 checkpoint

→ 可观测钩子(第5层)结束本 span, 上报

→ 调度循环读 state 决定下一个节点.....

→ 到"生成建议单"后 interrupt 挂起 (状态已落盘)

费曼总结：框架 = 状态容器定义"记什么"，调度循环定义"按什么顺序干"，工具运行时定义"怎么干实际活"，持久化钩子保证"干到哪记住哪"，可观测钩子保证"干了什么看得见"。五件齐了，Agent 才从 demo 变成系统。

4. 换框架 = 换语法，怎么用

理解共性件后的实战收益：

- 读新框架文档先找五层：**状态怎么定义？循环谁控制？工具怎么注册？持久化在哪？埋点在哪？五问过完，框架已入门。
- 迁移评估按层打分：**以 CTRM 点价需求为例，第 4 层（持久化+人审）权重最高——OpenAI Agents SDK 轻需求好用，但重状态场景 LangGraph 胜出（07 对比篇给完整决策树）。
- Java 平移：**Spring AI 的 ChatClient + Advisor 链对应第 2/3 层（链式编排+工具），Micrometer 对应第 5 层，但第 4 层缺位——Java 团队的典型架构是"编排层 Python LangGraph 服务化，业务层 Spring"或"Spring AI + 自建状态落库"。

【本节自测】

- 五层共性件分别是什么？（要点：状态容器/调度循环/工具运行时/持久化钩子/可观测钩子）
- 第 1 层的形态分化举两个极端？（要点：LangGraph 显式强类型 State ↔ CrewAI 隐式任务上下文）
- 调度循环的四种形态？（要点：图引擎、LLM 自主循环、角色剧本、对话轮转）
- 哪层是框架分野的核心，为什么？（要点：持久化钩子；决定能否长挂起人审/崩溃恢复）
- 手写 input() 对应框架什么件，多了什么能力？（要点：interrupt；可挂起数天、可改状态后恢复）
- MCP 与哪层共性件相关，现状如何？（要点：工具运行时；Spring AI 1.1+/LangChain4j 1.x 原生支持，2026 成事实标准）
- "换框架五问"是哪五问？（要点：状态/循环/工具/持久化/埋点怎么定义）

8. CTRM 点价 Agent 走一遍执行闭环, checkpoint 写在哪一步之间? (要点: 节点增量合并进 state 之后、下一节点调度之前)

04 · 进阶篇：持久化与 Human-in-the-Loop

【本节问题】① checkpointer 落盘的到底是什么，怎么组织？② interrupt 挂起几天再恢复，协议上怎么实现？③ 跨会话的长期记忆 (store) 怎么用？④ 多 Agent 的 subgraph 组合怎么做？

本篇是 05 实战篇的理论预备：先把 checkpointer、interrupt、store、subgraph 四大进阶件讲透，实战篇直接用代码收割。

1. Checkpointer: 每步自动存档

- 【一句话人话】图的每一步执行完，框架把整个状态快照写进后端存储；thread_id 是存档槽位号。
- 【生活类比】游戏存档系统：thread_id 是存档槽，每过一个节点自动存一次，死了读档，还能翻旧档重打 (time-travel)。

1.1 快照里有什么

每个 checkpoint 包含：channel values (当前状态各字段的值)、待执行任务、上一步来源节点、版本号与父 checkpoint 引用。checkpoint 之间构成链——这就是 time-travel 能回滚的原理 (沿着父指针回溯)。

1.2 三种后端选型

后端	场景	CTRM 对应
InMemorySaver	单测、demo	内存 Map
SqliteSaver	本地开发、单机工具	嵌入式 SQLite
PostgresSaver	生产 (并发、高可靠)	你们的生产 MySQL 角色 (LangGraph 官方提供 Postgres 后端；MySQL 后端无官方支持，截至 2026-09 未见权威披露，生产建议换 Postgres 或自实现 BaseCheckpointSaver)

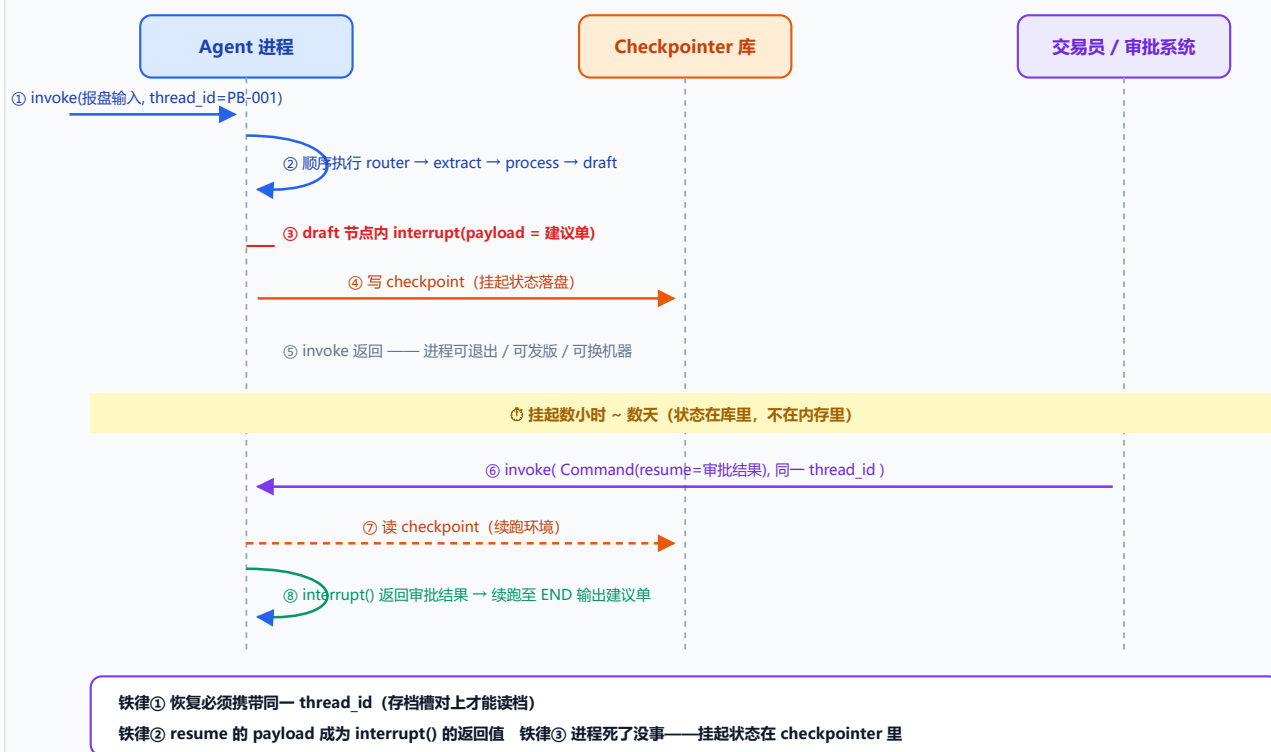
1.3 thread 与业务单据

- 一次会话/一单业务 = 一个 thread。点价 Agent 按"报盘单号"做 thread_id，天然把 Agent 状态和业务单据对齐——CTRM 工程师的心智：**thread 就是业务流水表的行，checkpoint 就是行版本**。

2. Interrupt: 人工中断与恢复

- 【一句话人话】让 Agent 在指定位置停下、把问题抛给人类，人回复后从停点继续——全程状态落盘，进程可以死、可以换机器。
- 【生活类比】信用卡大额交易人工审核：系统自动挡下可疑交易 (挂起)，审核员隔天上班处理 (恢复)，处理期间系统升级重启也不影响这单挂起。

009 • Human-in-the-Loop 中断恢复时序（点价建议单审批）



2.1 两种中断姿势

```

# 姿势 A: 编译时声明——进入某节点前必停
graph = builder.compile(checkpointer=saver, interrupt_before=["生成建议单"])

# 姿势 B: 运行时抛出一节点内部按业务条件动态停
from langgraph.types import interrupt, Command
def 生成建议单(state):
    proposal = build_proposal(state)
    review = interrupt({"proposal": proposal}) # 抛出中断, payload 给人看
    return {"confirmed": review["approved"]} # resume 时这里拿到人给的结果
    
```

- 姿势 A: 粗粒度, "这个节点永远要先审"。
- 姿势 B: 细粒度, "价格越限价带才审, 常规单自动过"——点价场景应选 B。

2.2 恢复协议（重点，面试高频）

```

第一次执行: graph.invoke(input, config={"configurable": {"thread_id": "PB-2026-001"}})
→ 跑到 interrupt 处停下, invoke 返回 (状态已落盘)

人工确认后恢复:
graph.invoke(Command(resume={"approved": True}), config=同一 thread_id)
→ 从中断点续跑, interrupt() 的返回值就是 resume 的 payload
    
```

三条铁律: 1. 恢复必须带同一个 thread_id——存档槽对上才读得到档。2. resume 的 payload 会成为 interrupt() 的返回值——把人的决定喂回节点内部。3. 中断期间进程可以死掉——这就是 checkpointer 的价值, 挂起状态在库里不在内存里。

2.3 点价业务映射

业务规则	实现选择
建议单必须交易员确认	节点内 interrupt(payload=建议单摘要)
低于 500 吨小额单自动过	interrupt 外包一层 if
交易员可以改价再确认	审批端改 payload 数据, resume 带回, 节点内覆盖建议单
确认记录留痕	resume payload 写审计表 + checkpoint 链天然可回溯

3. Store: 跨会话长期记忆

【一句话人话】Store 是按 namespace + key 组织的跨 thread 键值库, 存"客户偏好、历史结论"这类长期知识。

【生活类比】checkpointer 是本次交易的草稿纸, store 是客户档案柜——这单做完草稿纸扔掉, 档案柜里的客户偏好留着下次用。

```
from langgraph.store.memory import InMemoryStore
store = InMemoryStore()
# 写: 客户 C-1001 的点价偏好
store.put(("clients", "C-1001"), "pricing_pref", {"窗口": "09:00-10:30", "风格": "分批点价"})
# 读: 任何 thread 里都能取
pref = store.get(("clients", "C-1001"), "pricing_pref")
```

- 编译时 `graph.compile(checkpointer=saver, store=store)` 双件齐挂。
- 生产后端同样有 Postgres Store; 记忆写入的时机可以由"记忆节点"在每单结束后沉淀（跨会话学习, 007 多 Agent 篇的"经验沉淀"落地处）。

4. Subgraph: 多 Agent 组合 (联动 007)

【一句话人话】把一张编译好的图当作父图的一个节点, 父图管战略, 子图管战术。

【生活类比】集团把"点价业务"整个外包给一个子公司: 集团 (父图) 只发指令收结果, 子公司内部工序 (子图) 自理。

```
pricing_graph = builder.compile(checkpointer=saver)      # 点价子图
parent_builder.add_node("点价", pricing_graph)         # 子图当节点
```

- 状态对接: 父子图状态字段同名则自动传递; 不同名时用包装节点做映射 (避免父图状态被子图污染——**接口隔离**, 007 篇的通信边界原则)。
- 持久化归属: 子图可复用父图 checkpointer (同一 thread 连续存档), 也可独立挂 (子图自己就是完整可恢复单元)。
- 007 篇 supervisor 拓扑的 LangGraph 落地: 主管图 + 点价子图 + 套保子图 + 风控子图, handoff 就是主管图的条件边。

5. Java 生态对照段

本篇能力	Spring AI / LangChain4j 现状
状态快照	无内置; 自己用 Spring Data 落库 (等价于手写 003 的债)
人审中断	无原生 interrupt; 常规做法: 流程节点写"待审批"状态行 + 状态机 (如 Spring Statemachine), 回调续跑
长期记忆	LangChain4j ChatMemory 持久化扩展 / Spring AI MessageChatMemoryAdvisor, 均为对话级, 非跨业务知识库
subgraph	LangChain4j agentic 模块可组合 Agent; 无图级嵌套语义

(依据: fast.io 2026、CSDN 2026-07 对比文。) 结论: **Java 团队要么接受"人审部分自己建模成业务状态机", 要么把带 HITL 的编排层用 Python 服务化、Java 通过 REST/SSE 调用**——这是 07 对比篇给 Java 结论时的核心权衡。

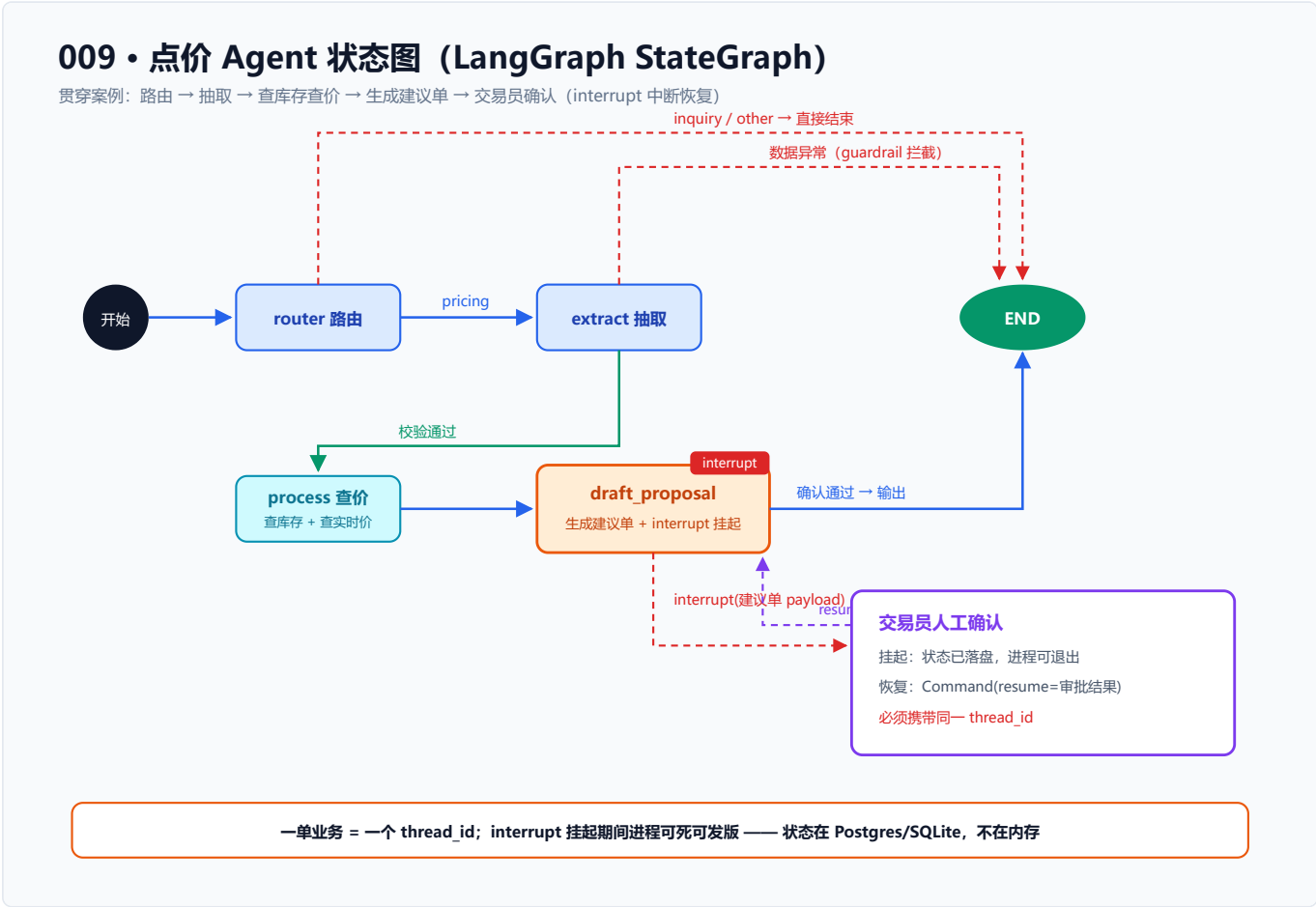
【本节自测】

1. checkpoint 快照包含什么? 为什么能 time-travel? (要点: 状态值+待执行任务+父指针; 链式结构可回溯)
2. 三种 checkpointer 后端与选型? MySQL 官方支持吗? (要点: 内存/SQLite/Postgres; MySQL 无官方支持, 截至 2026-09 未见权威披露)
3. interrupt 两种姿势与点价场景的正确选择? (要点: interrupt_before 声明式 vs 节点内 interrupt() 条件式; 选后者)
4. 恢复协议三要素? (要点: 同 thread_id + Command(resume=payload) + payload 成为 interrupt 返回值)
5. 中断期间进程死了为什么没事? (要点: 挂起状态在 checkpointer 库里不在内存)
6. store 与 checkpointer 的分工? (要点: 跨 thread 长期知识 vs 单 thread 会话历史)
7. subgraph 状态对接的坑? (要点: 字段同名自动传/异名需映射, 防父图状态污染)
8. Java 团队实现 HITL 的两条路? (要点: 业务状态机自建 / Python 编排服务化)

05 • 实战篇：LangGraph 重写点价 Agent（全代码）

【本节问题】① 点价 Agent 的 StateGraph 怎么定义？② 路由、抽取、查库存查价分别落成什么节点？③ 高危的建议单确认怎么用 interrupt 实现？④ checkpointer 与 stream 怎么接上？每段代码都标注"手写版对应物"，与 003 篇逐件对照。

1. 目标流程图



业务剧本（CTRM 场景）：客户报盘进来 → 路由判断类型 → 抽取要素（品种/数量/基差/点价窗口） → 查库存 + 查实时价 → 生成点价建议单 → 交易员人工确认（interrupt） → 确认后输出执行。

2. 全代码

运行环境：pip install -U langgraph langchain langchain-openai , OPENAI_API_KEY 环境变量。LangGraph 1.x API (2025-10-22 GA, 来源：aiwiki.ai / LangChain 博客) 。

2.1 状态定义

```
# ===== State: 整个 Agent 唯一的数据总线 =====
from typing import TypedDict, Literal, Annotated
from operator import add

class PricingState(TypedDict):
    raw_input: str # 客户原始报盘
    category: Literal["pricing", "inquiry", "other"] # 路由结果
    extracted: dict # 抽取要素: 品种/数量/基差/窗口
    stock_info: dict # 查库存结果
    price_info: dict # 查价结果
    proposal: dict | None # 点价建议单
    approval: dict | None # 人工确认结果
    audit_log: Annotated[list[str], add] # 审计日志 (reducer: 追加合并)
```

手写版对应物: 003 篇的 `state = {}` dict + 层层传参。框架差异: 字段显式可校验; `audit_log` 用 `Annotated[list, add]` 声明 reducer, 并行节点写日志不会互相覆盖。

2.2 节点一: router (路由)

```
# ===== 节点 router: 报盘类型分类 =====
from langchain.chat_models import init_chat_model

llm = init_chat_model("openai:gpt-4o-mini") # 模型无关, 可换任何 provider

ROUTER_PROMPT = """你是贸易助理。判断客户输入属于哪类:
- pricing: 报盘/点价请求
- inquiry: 仅询价咨询
- other: 无关内容
只输出一个词。"""

def router_node(state: PricingState) -> dict:
    resp = llm.invoke([("system", ROUTER_PROMPT), ("human", state["raw_input"])])
    cat = resp.content.strip()
    if cat not in ("pricing", "inquiry", "other"):
        cat = "other" # guardrail: 越界值兜底
    return {"category": cat, "audit_log": [f"router->{cat}"]}
```

手写版对应物: 003 篇 `route()` 里的 `if/elif` 关键词匹配。框架差异: 路由决策交给 LLM 更稳, 兜底校验 (guardrail 思想, 02 概念卡 8) 留在节点内; 分支的**走向**不写在这里, 而是声明为条件边 (见 2.5)。

2.3 节点二：extract（要素抽取）

```
# ===== 节点 extract: 结构化抽取 + 校验 =====
from pydantic import BaseModel, Field

class Extracted(BaseModel):
    # 结构化输出 schema
    commodity: str = Field(description="品种, 如 豆粕M2605")
    quantity: float = Field(description="数量 (吨)")
    basis: float = Field(description="基差报价")
    window: str = Field(description="点价窗口, 如 2026-09-20~2026-09-30")

EXTRACT_PROMPT = "从报盘中抽取品种、数量、基差、点价窗口。缺失字段给默认值并注明。"

def extract_node(state: PricingState) -> dict:
    resp = llm.with_structured_output(Extracted).invoke(
        [("system", EXTRACT_PROMPT), ("human", state["raw_input"])]
    )
    data = resp.model_dump()
    ok = data["quantity"] > 0 # 业务 guardrail
    return {
        "extracted": data,
        "audit_log": [f"extract ok={ok} {data['commodity']}"],
    }
```

手写版对应物：003 篇手拼 JSON prompt + 手写 json.loads + try/except。框架差异：with_structured_output 由 LangChain 统一处理 schema 注入与解析重试；Pydantic 校验挡住脏数据——第二层（控制流正确性）的收益开始显现。

2.4 节点三：process（查库存 + 查价，工具运行时）

```
# ===== 节点 process: 工具调用（查库存、查价）=====
from langchain_core.tools import tool

@tool
def query_stock(commodity: str) -> dict:
    """查询某品种的可用库存。"""
    return {"commodity": commodity, "available_tons": 1200} # 实际接 CTRM 库存接口

@tool
def query_price(commodity: str) -> dict:
    """查询某品种当前盘面价。"""
    return {"commodity": commodity, "last_price": 3120.0} # 实际接行情接口

def process_node(state: PricingState) -> dict:
    c = state["extracted"]["commodity"]
    stock = query_stock.invoke({"commodity": c}) # schema 校验 + 统一异常包裹
    price = query_price.invoke({"commodity": c})
    return {
        "stock_info": stock,
        "price_info": price,
        "audit_log": [f"process stock={stock['available_tons']} price={price['last_price']}"],
    }
```

手写版对应物：003 篇手写 TOOLS 描述表 + 手拼 function-calling JSON + 每个工具各写 try/except。框架差异：@tool 装饰器自动生成 schema；tool.invoke() 统一入口（超时/重试策略可全局配）——五层共性件里的“工具运行时”（03 原理篇第 3 层）。

2.5 条件边：路由分发与质量闸门

```
# ===== 条件边 =====
def route_after_router(state: PricingState) -> str:
    if state["category"] == "pricing":
        return "extract"          # 报盘 → 抽取
    return "__end__"             # 咨询/无关 → 直接结束（生产中可转客服子图）

def route_after_extract(state: PricingState) -> str:
    q = state["extracted"]["quantity"]
    if q <= 0 or q > 100000:
        return "__end__"         # 异常数据：拒绝（生产中应转人工）
    return "process"
```

手写版对应物：003 篇主循环里的 `if intent == "pricing": ... elif ...`。框架差异：分支从“代码流”变成“图上线”——加一条新报盘类型只改映射表，不动已有节点。这正是 01 问题篇第二层收益：**控制流声明化**。

2.6 节点四：draft_proposal（生成建议单 + interrupt 人审）

```
# ===== 节点 draft_proposal: 建议单 + 人工确认 =====
from langgraph.types import interrupt, Command

LIMIT_BAND = (2900.0, 3400.0)          # 限价带（业务规则）

def draft_proposal_node(state: PricingState) -> dict:
    e, p = state["extracted"], state["price_info"]
    suggest = round(p["last_price"] + e["basis"], 2)
    proposal = {
        "commodity": e["commodity"], "quantity": e["quantity"],
        "suggest_price": suggest, "window": e["window"],
        "within_band": LIMIT_BAND[0] <= suggest <= LIMIT_BAND[1],
    }
    # 业务规则：越限价带的单子必须人工确认；常规单也按公司制度走人审
    review = interrupt({                  # ← 挂起！状态已落盘，进程可死
        "type": "proposal_review",
        "proposal": proposal,
        "hint": "回复 {\\"approved\\": bool, \\"operator\\": 姓名}",
    })
    # resume 后从这里继续，review 就是人工提交的 payload
    final = {**proposal,
        "suggest_price": review.get("new_price", proposal["suggest_price"]),
        "approved_by": review["operator"]}
    return {"proposal": final, "approval": review,
        "audit_log": [f"proposal approved={review['approved']} by {review['operator']}"]}
```

手写版对应物：003 篇的 `input("请交易员确认(y/n): ")`。框架差异是质变：`input()` 阻塞进程、重启即丢；`interrupt()` 挂起在 `checkpoint` 里，**进程可以死、可以部署升级、可以换机器恢复，交易员甚至三天后批——这就是 04 进阶篇的恢复协议，01 问题篇第三层（最值钱层）的落地。**

2.7 组装: StateGraph + checkpoint + compile

```
# ===== 组装与编译 =====
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.sqlite import SqliteSaver

builder = StateGraph(PricingState)
builder.add_node("router", router_node)
builder.add_node("extract", extract_node)
builder.add_node("process", process_node)
builder.add_node("draft_proposal", draft_proposal_node)

builder.add_edge(START, "router")
builder.add_conditional_edges("router", route_after_router,
                              {"extract": "extract", "__end__": END})
builder.add_conditional_edges("extract", route_after_extract,
                              {"process": "process", "__end__": END})
builder.add_edge("process", "draft_proposal")  # process 完固定走建议单
builder.add_edge("draft_proposal", END)

# 持久化: SQLite 单机档; 生产换 PostgresSaver (04 进阶篇 1.2 节)
saver = SqliteSaver.from_conn_string("pricing_agent.db")
# interrupt_before=["draft_proposal"] 可选: 若想"进节点前必停"用姿势 A;
# 本例在节点内按业务条件 interrupt (姿势 B), 更细粒度
graph = builder.compile(checkpointer=saver)
```

手写版对应物: 003 篇的 `while step < MAX_STEPS`: 主循环 + 函数调用链。框架差异: 执行顺序全部声明在图上, `recursion_limit` 由引擎兜底 (默认 25), 不存在忘写上限死循环烧 token 的坑。

2.8 运行: invoke → 中断 → resume → stream

```
# ===== 第一次执行: 跑到 interrupt 自动停 =====
config = {"configurable": {"thread_id": "PB-2026-001"}}  # thread_id=报盘单号
result = graph.invoke({"raw_input": "客户A报: 豆粕M2605 2000吨, 基差+80, 9月20到30日点价"},
                      config)

print(result["audit_log"])
# ['router→pricing', 'extract ok=True 豆粕M2605', 'process stock=1200 price=3120.0']
# 建议单已生成, 挂在库里等审批。此时进程可以退出、可以发版。

# ===== 交易员确认后恢复 (可能发生在几天后、另一台机器上) =====
from langgraph.types import Command
final = graph.invoke(Command(resume={"approved": True, "operator": "张贸易"}), config)
print(final["proposal"])
# {'commodity': '豆粕M2605', 'quantity': 2000.0, 'suggest_price': 3200.0,
#   'window': '2026-09-20~2026-09-30', 'within_band': True, 'approved_by': '张贸易'}

# ===== 流式: 给前端 Vue 页面推实时进度 (SSE 对接) =====
for chunk in graph.stream(
    {"raw_input": "客户A报: 豆粕M2605 2000吨, 基差+80, 9月20到30日点价"},
    config, stream_mode="updates"):
    for node_name, patch in chunk.items():
        print(f"[{node_name}] 完成, 更新字段: {list(patch.keys())}")
# [router] 完成, 更新字段: ['category', 'audit_log']
# [extract] 完成, 更新字段: ['extracted', 'audit_log'] ...
```

手写版对应物: 003 篇"一次性函数返回"。框架差异: `updates` 流让前端能画进度条 ("已抽取→已查价→待审批"), `token` 级流 (`stream_mode="messages"`) 可做打字机效果——02 概念卡 12 的三个粒度在此对号。

3. 代码 ↔ 工程件总账

代码段	工程件	03 原理篇对照行
2.1 State	状态容器 (reducer)	表行 1
2.2 router + 2.5 条件边	调度循环 (分支)	表行 2、5
2.3 extract (structured output)	guardrail + 状态容器校验	表行 2、4
2.4 @tool	工具运行时	表行 4
2.6 interrupt	人审中断	表行 6
2.7 SqliteSaver	持久化钩子	表行 7
2.8 stream	可观测/推送	表行 8、9

4. 常见坑

1. **thread_id 复用**: 用旧 thread_id invoke 新输入会"接着上次状态跑", 点价单务必一单一 thread。
2. **interrupt 后重复 resume**: 一次中断只消费一次 resume; 重复提交同 payload 会报"无待恢复中断"。
3. **reducer 忘写**: 并行节点写同一 list 字段不加 `Annotated[... , add]` 会直接覆盖而非合并。
4. **SQLite 并发锁**: SqliteSaver 单机并发差, 多实例部署必须 PostgresSaver。
5. **interrupt 放在工具调用节点内**: 工具已执行的部分不可撤销, 中断点应设在"生成动作之后、执行动作之前"。

5. Java 生态对照段

同一段流程的 Java 写法骨架 (伪代码对照, 非完整可跑):

```
// Spring AI 1.x 风格: 无图编排, 用 Advisor 链 + 手写流程
ChatClient client = ChatClient.builder(chatModel)
    .defaultAdvisors(new MessageChatMemoryAdvisor(chatMemory)) // 记忆
    .defaultTools(new StockTools()) // 工具 (@Tool 注解)
    .build();
// 路由/分支/中断: 自己写 — @Transactional 状态行 + Spring Statemachine 实现人审挂起
```

- LangChain4j 的 `langchain4j-agentic` 模块提供 Workflow 编排 (顺序/并行/循环), 比 Spring AI 显式, 但同样**没有内置 checkpoint/interrupt** (依据: fast.io 2026、CSDN 2026-07)。
- 工程落地建议: Java 团队可将本节 Agent 作为 Python 微服务, 经 REST 暴露"提交报盘 / 查询状态 / 提交审批"三个接口, CTRM 主系统 (Java) 调用——人审挂起语义完全由 LangGraph 承担。

【本节自测】

1. PricingState 里 `Annotated[list[str], add]` 是什么, 为什么需要? (要点: reducer 声明; 并行写合并而非覆盖)
2. 路由决策为什么放节点内、分支走向放条件边? (要点: 决策=LLM/逻辑, 走向=图声明; 关注点分离)
3. `with_structured_output` 替代了手写版的哪三件事? (要点: schema 注入、输出解析、重试)
4. interrupt 的 payload 在恢复时去哪了? (要点: 成为 interrupt() 的返回值, 节点内消费)
5. 为什么建议单节点选姿势 B (节点内 interrupt) 而不是 interrupt_before? (要点: 按业务条件细粒度人审)
6. thread_id 用报盘单号的好处? (要点: Agent 状态与业务单据对齐, 一单一档)
7. stream_mode="updates" 推的是什么? (要点: 每节点完成的增量 patch; token 级用 messages)
8. 五个坑里哪个最可能在你们 CTRM 环境先爆? (要点: 开放; SQLite 并发/线程复用是高概率答案)

06 • 生产篇：可观测、评估与部署

【本节问题】① Agent 的 trace 怎么看、接哪个平台？② 框架自带的评估能力和 006 篇评估集纪律怎么结合？③ LangGraph 应用有哪些部署形态，怎么选？④ 版本升级有哪些已知的坑？

1. 可观测：trace 是 Agent 的事故现场

【一句话人话】生产上 Agent 出问题，你唯一能依赖的就是 trace——它记录每次执行每一步的输入、输出、token、耗时。

【生活类比】生产事故回查 = 查网关 access log + 应用 log + DB 慢查询三件套；Agent 的 trace = 这三件套合体的"时间旅行录像带"。

1.1 平台选型

平台	特点	适合
LangSmith	LangChain 同门，trace + 评估 + 数据集 + 部署一体化，商业 SaaS	已用 LangGraph 全家桶
Langfuse	开源可自托管，OTel 兼容，trace + eval + prompt 管理	数据不出内网（贸易公司常见合规要求）
OTel 直连	标准协议，接 Datadog/SkyWalking/Jaeger	已有 APM 体系的企业

LangGraph 每个节点执行天然是一个 span（03 原理篇第 5 层），接 LangSmith 只需设 `LANGSMITH_TRACING=true` + API key；接 Langfuse 用其 LangGraph callback 集成。CTRM 建议：**客户报盘数据涉密，优先 Langfuse 自托管。**

1.2 CTRM 的 trace 实践清单

- 1. **thread_id 进 trace 元数据**：报盘单号 PB-2026-001 能直接搜到完整链路——审计合规刚需。
- 2. **关键节点打业务标签**：router 结果、建议单价格带内外，作为 trace 的 tag，便于聚合统计"越限价带单占比"。
- 3. **token 成本按业务线分摊**：点价/套保/咨询分 namespace 记账。
- 4. **告警**：单次执行 span 数突增 = 疑似循环失控；interrupt 挂起超 24h = 审批积压。

2. 评估：框架 eval + 006 评估集纪律

框架给你**评估工具**，006 篇给你**评估纪律**——两者结合才有效。

2.1 LangGraph 侧的评估抓手

- **LangSmith Datasets**：把评估集（输入 → 期望）存为数据集，跑实验对比 prompt/模型版本，自动打分。
- **Langfuse Evals**：同样支持数据集 + LLM-as-judge + 人工标注队列。
- **单节点测试**：LangGraph 节点是普通函数——`extract_node` 可以不启动整图直接单测（框架化带来的可测性红利，01 问题篇）。
- **time-travel 回归**：改了 prompt 后从历史 checkpoint 分叉重跑，对比新旧结果——这是 checkpointer 附赠的独特评估手段。

2.2 与 006 篇纪律的结合

006 篇纪律	在框架下的落地
评估集进版本库、与代码同评审	数据集导出 JSON 入 git；LangSmith/Langfuse 只做执行载体
线上样本回流	trace 按规则筛（如人工否决的单）自动进评估集候选池

006 篇纪律	在框架下的落地
分层指标	节点级（抽取准确率） / 全链路级（建议单采纳率） / 成本级（token/单）
禁止线上直接试错	用数据集 + 实验（experiment）对比，胜出才发布

优先级要点：先建"抽取准确率"这一个指标并持续积累（成本最低、价值最高），再扩全链路指标。

3. 部署：三种形态

【一句话人话】Agent 服务的特殊之处：它有状态（checkpointer）、有长挂起（interrupt），部署方案必须先回答"状态放哪、挂起谁兜底"。

3.1 形态对比

形态	是什么	适用
LangGraph Server / LangSmith Deployment	官方托管/企业部署：自带持久化队列、Cron、双写 API（来源：LangChain 融资公告，原 LangGraph Platform 并入 LangSmith 部署）	想最快上线、接受 SaaS 或私有化报价
自托管 langgraph-cli / langgraph dev 容器化	官方开源 server 镜像打进自己的 K8s, Postgres 自管	数据合规、运维自主
作为库嵌入 FastAPI	graph 对象直接在自家 Python 服务里跑	只需轻并发、想复用已有服务框架

3.2 CTRM 推荐架构

Vue 2.7 前端 —SSE—> Java CTRM 主系统 —REST—> Python Agent 服务(FastAPI 嵌入 graph)
├ PostgresSaver（状态/挂起）
└ Langfuse 自托管（trace）

- Agent 服务无状态化：进程可随时重启，挂起的单在 Postgres 里——这是 04/05 篇持久化件在生产架构里的兑现。
- 人审闭环：Java 侧审批通过后调 Python 的 resume 接口（05 实战篇 2.8 的 `Command(resume=...)` 包成 REST）。
- 发布策略：评估实验胜出的 prompt/图定义作为版本化配置发布，回滚 = 切回上一版本配置。

4. 版本升级坑

- LangChain 1.0 断代**：2025-10-22 起 `langgraph.prebuilt` 弃用，功能迁入 `langchain.agents`；官方承诺 1.x 内无破坏性变更（来源：aiwiki.ai / LangChain 博客）。升级前先查弃用清单。
- 锁版本**：Spring AI 曾在小版本间破坏 API（fast.io 2026 指出须显式 pin 版本）；Python 侧同样建议 lock（uv/pip-tools），LangGraph 1.2.x 的命令式 API 等（DeltaChannel）只在最新 minor。
- checkpoint 序列化兼容**：升级可能改状态序列化格式——升级前确认 checkpointer 后端兼容矩阵，跨大版本升级先在新环境回放历史 thread 验证。
- 模型 API 变动**：`init_chat_model` 隔离了部分变动，但 provider 侧（如工具调用参数格式）仍可能波及，评估集回归是安全网。

【本节自测】

- trace 与 span 在排查时的用法？（要点：trace=全链路录像，span=分段；thread_id/报盘单号作为检索键）

2. LangSmith vs Langfuse 怎么选，贸易行业倾向谁，为什么？（要点：一体化商业 vs 开源自托管；数据合规倾向 Langfuse）
3. 框架评估工具和 006 纪律的分工？（要点：工具=执行载体，纪律=数据集版本化/回流/分层/禁线上试错）
4. time-travel 对评估的独特价值？（要点：历史 checkpoint 分叉重跑做回归对比）
5. 三种部署形态与选择依据？（要点：托管 server / 自托管容器 / 嵌入库；合规与运维自主性决定）
6. CTRM 架构里"挂起的单为什么不怕重启"？（要点：PostgresSaver 落盘，进程无状态）
7. LangChain 1.0 的主要迁移点？（要点：langgraph.prebuilt 弃用迁入 langchain.agents；1.x 承诺无破坏性变更）
8. 生产前必须做的两道回归？（要点：评估集实验胜出才发布 + 升级后历史 thread 回放验证）

07 · 对比篇：框架选型决策

【本节问题】① 要不要上框架的判断标准是什么？② 六个主流框架在各工程件上的真实差距？③ 轻需求、重状态、Java 团队分别选谁？④ 决策树怎么走？

1. 先问要不要框架

框架不是信仰，是工程件采购。三连问（对照 01 问题篇三层收益）：

- 1. 要不要崩溃恢复 / 长挂起人审？要 → 框架（checkpointer/interrupt 层手写代价最高）；不要 → 手写循环完全够（003 篇就是证明）。
- 2. 流程会不会频繁变？会（新报盘类型、新审批规则）→ 框架声明式控制流值得；一次性脚本 → 不值得。
- 3. 要不要全链路可观测/审计？合规要求 → 框架 trace 钩子白送；demo → print 够了。

三问有两个以上“要”，上框架；否则保持 003 篇手写版，别为炫技付学习成本。

2. 六框架大表（2026-09 现状）

009 · 六框架选型对比（2026-09 现状）					
同一副五层骨架的不同取舍 —— 版本与现状均有来源标注（见 07 对比篇）					
LangGraph 1.0 GA 2025-10-22 · 1.2.x 迭代中 显式状态图：State + 节点 + 条件边 checkpointer + interrupt 内置 优：重状态 / HITL / 审计首选 劣：学习曲线陡 · 图思维门槛 背书：Uber · LinkedIn · Klarna	OpenAI Agents SDK 2025-03 发布 · 替代实验性 Swarm 轻量四件套：tool · handoff guardrail · tracing 优：最轻量 · 官方出品 · 上手快 劣：无内置持久化 · 锁 OpenAI 生态 适合：单 Agent + 工具的轻需求	CrewAI 1.15.x · 宣称 1200 万+ 日执行 Role / Goal / Backstory 角色团队 任务按角色分配 · 隐式上下文 优：起步最快 · 配置最少 劣：无原生 checkpoint · 控制流弱 常见路径：原型用 CrewAI → 迁 LangGraph	AutoGen / AG2 → MAF 维护模式 · 并入 Microsoft Agent Framework MAF 1.0 GA 2026-04-03 (.NET/Python) 对话消息流多 Agent 鼻祖 · actor 化 优：group chat 协商范式成熟 注意：AutoGen 本体不再加新功能 建议：新项目走 MAF / 老项目看迁移指南	Spring AI 1.0 GA 2025-05 · 2.0 GA (Boot 4 + A2A) ChatClient + Advisor 链 · @Tool Micrometer / OTel 原生可观测 优：Spring 全家桶零成本融合 劣：编排 / HITL 仍在追赶 把 LLM 当成新一代 JDBC 的官方路线	LangChain4j 1.15 ~ 1.18 · agentic 模块 AI Services 接口声明式 · Workflow MCP 原生 · 框架中立 (Quarkus 可用) 优：Java 社区最活跃 · Provider 最广 劣：无内置持久化中断 适合：不绑 Spring 的 Java 团队
结论：轻需求 → OpenAI Agents SDK 重状态 → LangGraph Java 团队 → Spring AI (重 HITL 配 Python 编排服务化)					
版本现状来源：aiwiki.ai (2026-09 检索) · CSDN 六框架对比 (2026-08) · fast.io Java 框架榜 (2026) · 守株阁 (2026-05) · LangChain 融资公告					

维度	LangGraph	OpenAI Agents SDK	CrewAI	AutoGen/AG2	Spring AI	LangChain4j
版本现状	1.0 GA(2025-10-22), 1.2.x 迭代中	2025-03 发布, 替代 Swarm	1.15.x	维护模式 (并入 Microsoft Agent Framework, MAF 1.0 GA 2026-04-03)	1.0 GA(2025-05), 2.0 GA	1.15~1.18
抽象模型	显式状态图 (TypedDict State)	Agent + Run loop + handoff	Role/Goal/Backstory 角色团	对话消息流 (actor 化 0.4)	ChatClient + Advisor 链	AI Services 接口声明 + agentic Workflow
持久化/恢复	内置 checkpointer (内存/SQLite/Postgres) + time-travel	无内置 (需外接)	无原生 checkpoint	无内置	无内置	无内置
人工中断 HITL	一等公民 (interrupt/resume)	无原生	有限	会话级人工介入	无原生	无原生
可观测	LangSmith/Langfuse/OTel	内置 tracing (OTel 兼容)	内置监控+第三方	0.4 内置调试监控	Micrometer/OTel 原生	OTel 支持
MCP 支持	经 LangChain 生态	支持	支持	支持	1.1+ 原生	1.x 原生
多 Agent	subgraph/supervisor 手搭	handoff 原语	原生 crew/hierarchy	原生强项 (group chat)	基础	agentic 模块 (Workflow/A2A)
语言	Python / JS	Python / JS	Python	Python / .NET	Java (Boot 强绑定)	Java (框架中立, Quarkus/Spring 均可)
生产背书	Uber、LinkedIn、Klarna (来源: LangChain 博客/aiwiki.ai)	OpenAI 官方出品	1200 万+日执行 (来源: aiproplaybook)	社区治理, 历史项目多	Spring 生态	社区最活跃 Java AI 框架
学习曲线	陡 (图思维)	平缓	低 (角色配置)	中	低 (对 Spring 用户)	中

(来源汇总: aiwiki.ai 2026-09 检索; CSDN 《十六个主流AI应用开发框架全面对比》2026-08 版; 《LangChain4j vs Spring AI》2026-07; fast.io 2026; 守株阁 2026-05。)

3. 选型决策树

Q0 要不要框架？（第 1 节三连问，≥2 个“要”才继续）

|

Q1 团队主语言？

└ Java 团队

| └ 已是 Spring Boot 全家桶、需求以工具调用+RAG 为主 → Spring AI

| └ 需要多 Agent 编排/MCP 深度集成/不绑 Spring → LangChain4j

| └ 必须要持久化人审（点价式 HITL）→ 编排层 Python LangGraph 服务化，
Java 走 REST/SSE（02/04/05 篇结论）

|

└ Python 团队

└ Q2 状态重不重？（多步流程、挂起人审、审计回溯）

| └ 重状态 → LangGraph（本篇 05 实战的选择）

| └ 轻（单 Agent + 工具 + 简单 handoff，可接受 OpenAI 生态）→ OpenAI Agents SDK

|

└ Q3 是不是“角色分工”型任务？（研究小组、内容流水线）

| └ 是 → CrewAI（低配置快速起步，可靠性要求高了再迁 LangGraph——社区常见路径）

| └ 多 Agent 对话协商/已有 AutoGen 老项目 → 新项目走 Microsoft Agent Framework，
AutoGen 本体不再加新功能（维护模式）

4. 三条明确结论（锚点要求，可直接引用）

1. **轻需求用 OpenAI Agents SDK**：单 Agent、工具调用、简单转接，官方 SDK 四件套（tool/handoff/guardrail/tracing）覆盖，学习成本最低。代价：锁 OpenAI 生态、无内置持久化。
2. **重状态用 LangGraph**：多步流程、人审挂起、崩溃恢复、审计回溯——五层共性件全内置，生产背书最厚（Klarna 用其支撑 8500 万用户客服系统，来源：vantaige.io 2026-04）。代价：学习曲线陡、图思维门槛。
3. **Java 团队用 Spring AI**：与 Spring Boot 依赖注入/自动配置/Micrometer 零成本融合，1.0 GA 后风险可控，2.0 带来 Boot 4 + A2A（来源：fast.io）。代价：Agent 编排能力仍在追赶，重 HITL 场景需要“Python 编排服务化”混合架构或自建状态机。

5. CTRM 点价场景的最终建议

- **试点期（现在）**：Python LangGraph 跑点价 Agent（05 实战版），Java 主系统 REST 集成——验证业务价值。
- **规模化后（可选）**：若 HITL 需求被验证为“只需要简单审批流”，可评估迁 Spring AI + Spring Statemachine 自建持久化，消除双语言运维；若需要复杂编排（多子图、时间旅行审计），维持 LangGraph 编排层。
- **明确不推荐**：为了“技术栈统一 Java”而用 Spring AI 硬扛重状态编排——等于把 01 问题篇第三层（最值钱层）的债重新背回自己身上。

【本节自测】

1. 上不上框架的三连问？（要点：恢复/挂起人审？流程常变？可观测审计？）
2. 六框架里持久化+人审唯一内置的是谁？（要点：LangGraph；checkpointer + interrupt 一等公民）
3. OpenAI Agents SDK 的出身、四件套与代价？（要点：2025-03 替代 Swarm；tool/handoff/guardrail/tracing；锁生态无内置持久化）
4. AutoGen 2026 现状与新项目建议？（要点：维护模式并入 MAF（2026-04-03 GA）；新项目走 MAF）
5. Spring AI 与 LangChain4j 怎么二选一？（要点：Spring 全家桶/工具 RAG 为主→Spring AI；多 Agent/MCP/框架中立→LangChain4j）
6. “轻需求/重状态/Java 团队”三句结论？（要点：OpenAI Agents SDK / LangGraph / Spring AI）
7. CrewAI 的适用与社区常见迁移动作？（要点：角色分工型、快速起步；可靠性要求高后迁 LangGraph）

8. CTRM 点价场景试点期与规模化后的路线? (要点: LangGraph 服务化试点 → 验证后按 HITL 复杂度决定是否迁 Spring AI)

08 • 自测篇：题库与面试题

【本节问题】① 009 全章的概念能不能闭卷复述？② 面试官会怎么问框架问题？③ 实操题能不能独立完成？④ 哪些高频坑必须形成肌肉记忆？

用法：先遮住要点自答，再核对。分四组：概念题 / 原理题 / 实操题 / 面试实战题。

A. 概念题（对照 02 概念篇）

A1. StateGraph 为什么允许环？DAG 不行吗？ 要点：LLM 步数不定（工具可能调多轮），需循环直到终止条件；DAG 只能单向无环，表达不了 ReAct 迭代。

A2. 节点返回 dict 的语义？reducer 干什么？ 要点：返回增量 patch；reducer 定义同字段并发/多次写入如何合并（如 `Annotated[list, add]` 追加），默认覆盖。

A3. 条件边 vs Command(goto+update) 的选择？ 要点：条件边=图上显式分支（全局可审查，通用）；Command=节点内打包“更新+跳转”（分支与节点强耦合时）。

A4. checkpointer 和 interrupt 为什么是搭档？ 要点：interrupt 挂起后进程可死、可隔天恢复，靠的是状态已落盘；恢复=读档+续跑。

A5. handoff 与 subgraph 的区别？ 要点：handoff=运行时控制权移交（多 Agent 转接）；subgraph=编译期模块化复用（图当节点）。可组合：主管图用条件边实现 handoff 到子图。

A6. 短期记忆/长期记忆实现件？ 要点：短期=thread 内 state 历史（checkpointer 自动）；长期=Store（namespace+key，跨 thread）。

A7. trace/span 与 LangGraph 的天然对应？ 要点：一次图执行=trace；一个节点执行=span；thread_id 做检索键。

A8. streaming 三粒度与生产最常用？ 要点：token（打字机）/事件 updates（进度条）/状态；生产最常用 updates。

B. 原理题（对照 03 原理篇）

B1. 五层共性件？哪层最分野？ 要点：状态容器/调度循环/工具运行时/持久化钩子/可观测钩子；持久化层最分野（决定 HITL 与恢复能力）。

B2. 003 手写循环的哪些件在框架里是“能力从无到有”？ 要点：人工确认（input→interrupt）、断点恢复（无→checkpointer）、全链路日志（print→trace）；状态/路由/工具只是“表达力升级”。

B3. “换框架五问”？ 要点：状态怎么定义、循环谁控制、工具怎么注册、持久化在哪、埋点在哪。

B4. MCP 对应哪层共性件，2026 现状？ 要点：工具运行时；Spring AI 1.1+、LangChain4j 1.x 原生支持，事实标准。

C. 实操题（对照 05 实战篇）

C1. 手写：定义 PricingState 并让 audit_log 支持并行安全追加。 要点：`audit_log: Annotated[list[str], add]`。

C2. 手写：给“生成建议单”节点加入审——价格越限价带才挂起，常规单自动过。 要点：节点内 `if not within_band: review = interrupt({...})`；恢复 `Command(resume={...})`。

C3. 手写：编译图并支持崩溃恢复与流式进度。 要点：`compile(checkpointer=SqliteSaver.from_conn_string(...))`；`graph.stream(..., stream_mode="updates")`。

C4. 排错：同一 thread_id 二次 invoke 报“无待恢复中断”？ 要点：一次中断只消费一次 resume；或该 thread 已跑完，新任务应开新 thread。

C5. 排错：并行节点写 extracted 字段互相覆盖？ 要点：给字段声明 reducer；或重新设计避免并行写同字段。

C6. 迁移：把 05 的 SqliteSaver 换成生产可用后端。 要点：PostgresSaver；MySQL 无官方 checkpoint（截至 2026-09 未见权威披露），可自实现 BaseCheckpointSaver 或换 Postgres。

D. 面试实战题

D1. 面试官："你说用了 LangGraph，它解决了你什么问题？"（考察：是否真懂框架价值分层） 参考答：三层——样板收敛是顺带；控制流声明化（条件边上图）是效率；最关键的是 checkpoint+interrupt 给了"挂起数天的人审 + 崩溃恢复"，这在手写版里等价于自建消息队列+状态库+恢复协议。

D2. 面试官："LangGraph 和 CrewAI 怎么选？"（考察：抽象模型差异） 参考答：LangGraph 显式状态图、可控可审计、学习陡；CrewAI 角色隐喻、起步快、持久化弱。社区常见路径：CrewAI 原型 → 可靠性/审计要求出现后迁 LangGraph。

D3. 面试官："Java 团队怎么做带人工审批的 Agent？"（考察：JVM 生态短板认知） 参考答：两条路——①Spring AI + Spring StateMachine 自建挂起状态行（把 HITL 建模成业务状态机）；②编排层 Python LangGraph 服务化，Java 走 REST/SSE。当前 JVM 框架无内置 checkpoint/interrupt（2026-07 对比文），重状态场景不硬扛。

D4. 面试官："interrupt 的恢复协议说说？"（考察：细节深度） 参考答：中断时状态已写 checkpoint；恢复带同一 thread_id invoke `Command(resume=payload)`；payload 成为节点内 interrupt() 的返回值；进程可跨机器、跨时间恢复。

D5. 面试官："Agent 上线后怎么排查'昨天有一单建议单价格离谱'？"（考察：可观测体系） 参考答：报盘单号=thread_id 检索 trace → 看 extract 节点输入输出（抽取错？）→ 看查价 span（行情源错？）→ 看 draft_proposal 的限价带判断（规则错？）→ time-travel 分叉重放验证修复。

D6. 面试官："为什么 2026 年框架数量收敛了？"（考察：行业视野） 参考答：1.0 里程碑密集（LangGraph/LangChain 2025-10、MAF 2026-04、Spring AI 2025-05），AutoGen 进入维护模式，格局沉淀为图编排/官方 SDK/角色协作/Java 生态四个梯队；加上 MCP/A2A 标准化降低了框架间差异。

【本节自测】

1. A 组 8 题全对 → 02 篇过关；错 2 题以上回去重读概念卡。
2. B 组 4 题全对 → 03 篇过关；B1 答不出五层 = 重读 009-02 图。
3. C 组至少独立完成 C1~C3 → 05 篇过关；C4~C6 是生产排错肌肉记忆。
4. D 组能脱稿讲 3 题 → 具备面试表达；D4 的恢复协议是高频深挖点。
5. 终极检验：合上资料，向同事用 3 分钟讲清"003 手写版到 05 框架版，哪三件事是质变"。（要点：人审挂起、崩溃恢复、全链路 trace）